

Subtask Module Implementation – FastAPI

1. Subtask Model

- Created Subtask model with fields: id, title, description, status, task_id.
- Established relationships:
 - `task = relationship("Task", back_populates="subtasks")`
 - Task model has `subtasks = relationship("Subtask", back_populates="task", cascade="all, delete-orphan")`.

2. Pydantic Schemas

- `SubtaskIn` → for creating/updating subtasks.
- `SubtaskOut` → for responses, with `from_attributes=True` to support ORM models.

3. Handlers

- `create_subtask` → Create a subtask under a task.
- `update_subtask` → Update subtask fields; admin can update all, user can update only allowed fields.
- `delete_subtask` → Admin deletes subtasks.
- `get_task_subtask` → Retrieve subtasks based on role:
 - **Admin** → sees all subtasks of tasks they created.
 - **User** → sees subtasks only for tasks assigned to them.

4. Routes

- `POST /tasks/{task_id}/subtasks` → create subtask
- `PUT /subtasks/{subtask_id}` → update subtask
- `DELETE /subtasks/{subtask_id}` → delete subtask
- `GET /subtasks/{task_id}` → list subtasks for a task based on user/admin role

5. Access Logic

- **Admin**: Can see all subtasks of tasks they created and can modify all fields.
- **User**: Can only see subtasks of tasks assigned to them and can modify only allowed fields.

